

Federated Querying over Structured Databases and LLMs for Job-Market Analytics

Group No: 07

Shashank

Department of Computer Science
IIIT-Delhi
New Delhi, India
shashank22469@iiitd.ac.in

Akshat Wadhera

Department of Computer Science
IIIT-Delhi
New Delhi, India
akshat057@iiitd.ac.in

Abstract

We build a federated query system that answers hiring and market questions by combining relational data (jobs, skills, and candidates) with large language models (LLMs). Natural language inputs are parsed into structured SQL plans executed on PostgreSQL (jobs_db, candidates_db); optional LLMs augment results with brief advisories. The planner is *deterministic* (regex-based decomposition into a small set of tasks) with an optional LLM JSON-mode parser fallback. At production scale, we load 1.35M jobs, 2.48M skills, and 24.7M job-skill pairs. We also detail key engineering choices: we *use* staging+de-dup joins for fast bulk loads, and we *implement* Postgres-specific date/location handling. Our system couples these structured results with two important mechanisms. First is an evidence-based *candidate legitimacy score* that evaluates how well a candidate's claimed skills are supported by experience, projects, and transformer-extracted text skills, and second is jobs and skills clustering to help determining closely related jobs and skills.

1 Introduction

Natural language interfaces to databases (NLIDB) make analytics accessible without schema knowledge. Hiring analytics is a perfect fit: users ask open-ended questions ("top skills for backend"), yet the answers must be grounded in structured facts (jobs, candidates, skills). Our system accepts NL questions, decomposes them into SQL over PostgreSQL, and augments the final view with concise LLM guidance.

2 Motivation and Objectives

2.1 Project Objectives

- Build a reliable jobs/candidates warehouse in PostgreSQL.

- Implement a deterministic NL $\rightarrow$ SQL planner with clear, auditable templates.
- Federate across jobs_db and candidates_db when a query needs both.
- Use LLMs only as an *assistant* (parser fallback, brief advisories), and provide relevant contextual information based on the query.

2.2 Motivation For This Project

- **Real need:** Hiring stakeholders phrase needs in NL; answers must be precise, explainable, and up-to-date.
- **Clear boundary:** Databases for correctness; LLMs for usability/context. This reduces hallucinations.
- **Reproducibility:** Deterministic templates (no fuzzy skill merge) make results stable and auditable.

3 Data Collection and Schema

3.1 Data Sources

The structured and unstructured data is collected from:

- **Job Postings CSV** (jobs_postings_csv):
job_link, job_title, company,
job_location, search_city,
search_country.
- **Job Skills CSV** (jobs_skills_csv):
job_link + comma-separated job_skills.
- **Candidates (Hugging Face):** default datasetmaster/resumes split with personal info, experience, education, projects, languages, skills.
- **Unstructured Source (LLM—Gemini):** Google Gemini. Used to generate short 3-section *advisor notes*. The LLM *never* produces raw SQL.

3.2 (Database Schemas)

Jobs DB

- jobs(job_id, job_link, title, company, location, title_lc, location_lc, country, country_lc)
- skills(skill_id, name, name_lc)
- job_skills(job_id, skill_id); PK: (job_id, skill_id)

Candidates DB

- candidates(cand_id, name, email, phone, location, location_lc, country, summary, certifications)
- skills(skill_id, name, name_lc) (*shared vocabulary intent*)
- candidate_skills(cand_id, skill_id, source)
- experience(cand_id, company, title, location, location_lc, start_date, end_date, duration, responsibilities)
- education(cand_id, institution, degree, major, start_date, end_date, gpa, honors, accreditation)
- projects(cand_id, name, role, description, impact, url, technologies)
- languages(cand_id, language, level)

Sources & Curation

We ingest large CSVs of job postings and job-skill pairs (publicly shared corpus).

Cleaning steps.

- Normalize columns and add lowercase variants: title_lc, location_lc, country_lc.
- Strict type handling; drop duplicates by job_link.
- Cap per-job skills to 30 to avoid noisy long tails.
- Canonical skill merge *disabled*: small quality gain vs. complexity; we keep the pipeline simple, deterministic, and reproducible.

Attempted skill consolidation (why disabled).

We experimented with clustering/normalizing similar skills (string rules + embeddings). This only collapsed $\sim 60k$ of $\sim 2.4M$ job-skill pairs (low yield) and introduced ambiguity. To preserve auditability and stable counts, we retained deterministic canonicalization (lowercasing + punctuation strip) and turned off fuzzy merging.

4 Methodology

4.1 Sample Queries

It can do queries like this—mixing SPJ, aggregates, and federated matches (using only our project’s examples):

- **Federated readiness & hiring plan:** “i am looking for people to build my website, for what job roles should i hire for and top 10 potential candidates”
- **Top- k role skills (Aggregate):** “Top 5 skills for Data Engineer”
- **Jobs listing (SPJ):** “Jobs listing for software engineer in London”
- **Federated readiness for a specialty:** “Candidates SRE readiness”

The first and fourth are federated (learn role-skill set from jobs, then rank candidates by overlap/coverage/Jaccard). The second and third operate on the jobs side only. A short LLM advisory can be shown, but counts/lists always come from SQL. We have included examples 1 and 3, and a step by step flow below in the *Examples* section.

4.2 SQL Generation

Deterministic templates (no text-to-SQL model).

We use a small regex parser to map NL queries into one of six task templates and compile to audited PostgreSQL. The LLM *does not* generate SQL or numbers; it only helps steer ambiguous wording toward the correct template (task selection, role/skill/location normalization). In later submissions, **if feasible**, we plan to explore upgrading this steerer into a cautious SQL writer for niche cases, while keeping audited, hand-written templates as the primary path for correctness and reproducibility.

5 System Architecture

5.1 Structured Querying

Structured sources are queried via **PostgreSQL** directly (no live external APIs during query). We

bulk-ingest CSVs and the resumes dataset into materialized tables with indexes; queries hit these tables for predictable latency.

5.2 Hybrid Querying

We adopt a **hybrid** architecture:

- **Materialized core (Postgres):** jobs, skills, job_skills, and normalized candidate entities.
- **On-the-fly federation:** when needed, we execute two SQL plans (jobs-then-candidates) and merge results in the application layer (overlap/coverage/Jaccard).
- **LLM (unstructured) role:** used only to *guide* queries toward deterministic templates (task selection, synonym/typo resolution). It never returns SQL or numeric results.

6 Algorithm

The core federated query processing pipeline is shown below:

```
# Deterministic NL -> SQL -> Execute ->
  Summarize (+ optional advisory)
def FederatedQueryProcessor(query, cfg):
    # Step 1: Parse/classify into a fixed task
    schema
    parsed = regex_parse(query) or
    llm_steer_to_template(query)

    # Step 2: Compile to audited SQL plan(s)
    plan = to_sql(parsed, cfg) # single-source
    or two-source (federated)

    # Step 3: Execute against PostgreSQL
    result = execute_with_adapter(plan) #
    jobs_db and/or candidates_db

    # Step 4: Post-process (ranking/thresholds)
    # - readiness: overlap/coverage/Jaccard vs
    role-skill set
    # - experience: years >= min and optional
    role overlap gate

    # Step 5: Summarize (DB-grounded) +
    optional advisory bullets
    tips = gemini_advisor(parsed) # WHAT TO
    SCREEN FOR / RESULTS HIGHLIGHTS / BONUS
    return format_summary(parsed.task, result),
    tips
```

7 Task 2B - Using seperated databases and using LLM as data source

Task 2B part (a) — Decomposition and Federated SQL Execution

For Task 2(a), each of our five NL queries (Q1–Q5) is first mapped to a ParsedQuery object by the de-

terministic regex-based planner in analyzer.py. The planner assigns:

- a **task** label (e.g., candidate_readiness, top_skills_for_role, list_jobs_for_role, eligible_companies_for_candidate),
- the **mode** (SPJ vs. AGG),
- normalized **role**, **location**, **skill(s)**, and any **topk** and **min_years** hints,
- plus an llm_subquery field capturing the LLM prompt stub.

The to_sql function in decompose.py then compiles this ParsedQuery into one or more *audited* SQL statements and a small execution plan:

- **Single-source plans** (e.g., Q2, Q3, Q4, Q5) carry a single SQL string, parameters, and an optional post-processor.
- **Federated plans** (e.g., Q1: candidate_readiness) carry a pair (sql_jobs, sql_candidates) and a pair of parameter tuples.

We explicitly deploy jobs_db and candidates_db as two **separate PostgreSQL instances** on different IPs. The Python adapter holds two independent connection pools keyed by logical names ("jobs_db", "candidates_db"). During execution, the run() function:

1. uses the source field in the plan to select the correct DB instance(s),
2. issues the SQL to the corresponding IP/instance via adapter.run_sql(source, sql, params),
3. and, for federated tasks, merges the two result sets in memory (e.g., Jaccard/coverage computation over role-skill vs candidate-skill sets in Q1).

This ensures that every NL query in our Task 2 set is backed by a concrete, inspectable SQL plan, and that cross-source queries are *genuinely federated* across different physical database endpoints.

Task 2B part (b) — LLM Prompt Re-writing after Decomposition

After decomposition, we do not send the raw user question to the LLM. Instead, we derive a *compact, task-aware prompt* stored in `ParsedQuery.llm_subquery`. This field is constructed directly in `parse()` (and mirrored in the LLM JSON fallback `llm_parse()`) using lightweight templates such as:

- "Give brief interview prep pointers for '{role}'" for `candidate_readiness`,
- "Summarize emerging tools and trends for the role '{role}'" for `top_skills_for_role`,
- "Extract short advice for candidates applying to '{role}' roles" for `list_jobs_for_role`,
- "Give concise screening heuristics for eligibility to '{role}' roles" for `eligible_companies_for_candidate`.

In other words, the LLM is always called with:

1. a **sanitized, structured view** of the user intent (role, skills, experience, location),
2. and a **short, deterministic meta-prompt** (`llm_subquery`) tied to the internal task.

This satisfies Task 2B part (b): the “LLM-specific text prompt” is *derived after* analysis and decomposition, and is safe to pass to the open LLM without leaking raw SQL or confusing, multi-intent phrasing.

Task 2B part (c) — Integrating LLM Results with SQL Entities

Finally, in Task 2(c) we integrate the LLM output with the structured SQL results. The core pipeline (see Algorithm section) is:

1. Execute the SQL plan and compute a JSON-serializable `structured_result`:
 - e.g., for Q1: `{"role_skills": [...], "matching_candidates": [...]}`,
 - for Q3: a list of jobs with `job_id`, title, company, location,
 - for Q4/Q5: a ranked list of companies with overlap counts (`overlap_hits`).

2. Call the LLM with `llm_subquery` and, where appropriate, a brief structured context (e.g., top skills, role name, or number of candidates) via our `gemini_advisor(parsed)` helper.
3. Attach the free-text LLM response as an *advisory* field alongside the structured entities.

The final response object has the form:

```
{
  "structured_result": { ... SQL-backed
    entities ... },
  "advisor": "Short, task-specific LLM notes"
}
```

For example:

- In Q1, the advisor highlights what to screen for when interviewing the top candidates surfaced by the federated readiness computation.
- In Q2, it comments on emerging tools and trends given the top 5 skills for *Data Engineer*.
- In Q4 and Q5, it provides screening heuristics (e.g., “look for services experience, distributed systems, MLOps exposure”) that are *displayed next to* the ranked company list derived from SQL.

Thus, LLM output is always:

1. **anchored** to concrete, DB-derived entities (jobs, candidates, skills, companies),
2. **non-authoritative** (it never provides counts, IDs, or SQL),
3. and **additive**: it enriches the SQL results with human-readable guidance, satisfying Task 2(c) without compromising determinism or auditability.

We have update the examples in the examples section below

8 Innovation: Cluster-Aware Exploration and Evidence-Based Candidate Legitimacy

Beyond building a basic NLIDB over jobs and candidates, we introduce two concrete innovations that improve the quality and usefulness of answers:

1. **Job and Skill Clustering** — we cluster job roles and skills into latent groups to support exploratory queries such as “similar roles” and “related skills”, instead of relying only on exact string matches.

2. **Candidate Legitimacy Scoring** — we compute an evidence-based *legitimacy score* per candidate in $[0, 1]$ that measures how trustworthy their skill claims are, and use this score inside our federated planner to rank and filter candidates.

The first innovation addresses *semantic sparsity* on the jobs side: titles and skills are noisy and fragmented (“backend engineer”, “backend developer”, “server-side engineer”) and we want a higher-level representation.

The second addresses *trustworthiness* on the candidate side: many resumes list long skill sections with no supporting evidence in experience or projects. Together, these two components make our system more exploratory (via clusters) and more robust (via legitimacy scoring), while remaining fully auditable.

8.1 Innovation 1: Job and Skill Clustering

8.1.1 Problem and Motivation

Raw job titles and individual skills are extremely fine-grained. Simple string matching can tell us that “backend engineer” roles often mention Java and Spring, but it cannot answer:

- “What *other roles* tend to look like backend engineers in practice?”
- “If someone knows Kubernetes, what *related skills* commonly co-occur in real jobs?”

We therefore introduce clustering on top of jobs_db: one level for **roles** and one for **skills**. These clusters smooth over job-level and skill-level sparsity and enable semantic queries like `similar_roles_for_role` and `similar_skills_for_skill`. `:contentReference[oaicite:0]index=0`

8.1.2 Representation and Models

Role (job) clusters. We first build a compact text representation per job:

$\text{text}(j) = \text{title_lc}(j) \parallel \text{concatenated skill names},$

constructed via a SQL query that joins jobs, job_skills, and skills. This yields one low-ercase text field per job with the title and all associated skills. `:contentReference[oaicite:1]index=1`

We then apply a standard text embedding + clustering pipeline:

- **TF-IDF** over job texts using `TfidfVectorizer` with up to `max_features=30000` unigrams and `min_df=10` to ignore extremely rare tokens.
- **Dimensionality reduction** using `TruncatedSVD` to a dense 128-dimensional representation. This preserves the main variance directions while keeping the matrix small enough for clustering.
- **Clustering** with `MiniBatchKMeans` into $K_{\text{jobs}} = 100$ role clusters (default). We use large mini-batches (10,000) to handle hundreds of thousands of jobs efficiently on CPU.

Each cluster is summarized by:

- a *cluster id* (integer),
- a short text *label* (heuristically derived from frequent terms),
- and a list of *top terms* (most characteristic title/skill tokens).

Skill clusters. For skills, the semantics come from which jobs they appear in. We therefore build a skill-by-job co-occurrence representation and perform clustering in that space.

To keep the matrix tractable but still capture most of the signal, we split the skills into a high-frequency “head” and a long “tail”:

1. We first select the top N most frequent skills (by job count). In our experiments we use $N = 10\,000$ as the “head” vocabulary. Although these 10 000 skills form only about 1% of all distinct 1.2% million skill strings in the raw corpus, they already participate in 16 263 082 job–skill pairs out of 24 711 819 total pairs (about two-thirds of all co-occurrences). This justifies using the top 10 000 skills as the main clustering force: they cover most of the real usage while keeping the matrix small.
2. For these head skills we build a sparse binary matrix $M \in \{0, 1\}^{|S_{\text{head}}| \times |\mathcal{J}|}$ where $M_{s,j} = 1$ if skill s appears in job j .
3. We L2-normalize each skill vector (row) so that clustering is driven by cosine similarity of job distributions.

4. We cluster the normalized vectors into $K_{\text{skills}} = 50$ skill clusters using MiniBatchKMeans.

Intuitively, two head skills fall into the same cluster if they tend to appear in similar sets of jobs; for example, {pandas, numpy, scikit-learn} or {kubernetes, docker, helm}.

To avoid ignoring rare skills, we then *project tail skills onto these head-defined clusters*. For each tail skill (not in the top- N set), we look at the jobs where it appears, aggregate which head clusters those jobs belong to, and assign the tail skill to the cluster with the highest co-occurrence mass. In this way, all skills receive a `skill_cluster_id`, but only the frequent head skills influence the cluster centroids; tail skills are attached afterwards, preserving nuance without blowing up the clustering cost.

8.1.3 Offline Pipeline and Storage

The clustering logic runs in a dedicated offline script which connects directly to `jobs_db`. It writes out four CSV artifacts under a cluster directory:

- `role_cluster_info.csv`: one row per role cluster with `id`, `label`, and top terms.
- `job_role_clusters.csv`: mapping from each `job_id` to a `role_cluster_id`.
- `skill_cluster_info.csv`: one row per skill cluster with `id`, `label`, and cluster size.
- `skill_cluster_membership.csv`: mapping from each `skill_id` (both head and tail) to one `skill_cluster_id`.

These CSVs are then bulk-loaded into `jobs_db` as regular tables:

- `role_clusters(role_cluster_id, label, top_terms, ...)`
- `job_role_clusters(job_id, role_cluster_id)`
- `skill_clusters(skill_cluster_id, label, size)`
- `skill_cluster_membership(skill_cluster_id, skill_id)`

so that our planner can treat cluster information just like any other relational dimension.

8.1.4 Integration into the Query Planner

We extend the deterministic planner with two new tasks:

- **similar_roles_for_role**: given a base role name, we:
 1. Find the dominant `role_cluster_id` for jobs whose `title_lc` matches the base role.
 2. Within that cluster, aggregate other titles by frequency.

The executor returns the cluster label, its top terms, and a ranked list of similar role names. This directly powers queries like “Roles similar to backend engineer”.

- **similar_skills_for_skill**: given a skill string, we:
 1. Identify the best-matching `skill_id` by substring match.
 2. Look up its `skill_cluster_id` via `skill_cluster_membership`.
 3. List other skills in that cluster, ranked by how many jobs they appear in.

Because tail skills are also mapped into these clusters, this works even for relatively rare skills as long as they co-occur with some head skills in real jobs. The executor returns the cluster label, size, and a list of related skills. This supports queries like “What skills are related to Kubernetes?”.

Because all of this is implemented via explicit tables and SQL, the clustering innovation remains fully auditable: we can always show which job/skills led to a cluster and why a particular role or skill was returned as “similar”.

8.2 Innovation 2: Candidate Legitimacy Scoring and Trustworthiness

8.2.1 Problem and Motivation

Candidate resumes often contain long skill lists that may be copy-pasted or outdated. If we treat these skills as ground truth, any matching or readiness score will be misleading. We want a principled way to answer:

- “How much *evidence* is there that this candidate really uses the skills they claim?”

- “Can we down-weight or filter profiles that list skills but never mention them in experience, projects, or education?”

To address this, we introduce a **candidate skill-legitimacy score** $L \in [0, 1]$ and store it alongside each candidate. This score is evidence-based, not purely heuristic, and is computed once during ingestion using both structured resume fields and a transformer-based skill extraction model.

8.2.2 Evidence Extraction and Model

For each raw resume record, we extract three views:

1. **Claimed skills:** structured skills under `skills.technical.*.name`. These are what the candidate explicitly claims.
2. **Structured contexts:** technology-related tokens from:
 - `experience.technical_environment` (technologies, tools, methodologies, databases, operating systems),
 - project technologies,
 - and selected education fields and relevant coursework.
3. **Free-text body:** concatenation of responsibilities, project descriptions, and other textual fields into a single long string.

We run a **transformer-based skill NER model**, Nucha/Nucha_ITSkillNER_BERT, over the free-text body. This model tags spans corresponding to IT skills (e.g., “Docker”, “Kubernetes”, “PostgreSQL”). We canonicalize all skill strings (lowercasing, trimming) and count occurrences:

$\text{text_skill_counts}[s] = \text{number of times skill } s \text{ is detected in text.}$

For each claimed skill s , we define three evidence counts:

1. **Structured evidence.** This counts how many times s appears in the structured resume fields (e.g., experience environments, project technologies, relevant coursework), after canonicalisation (lowercasing, trimming) of each token. We denote this by $N_{\text{structured}}(s)$ and use

$$\text{structured_evidence}(s) = N_{\text{structured}}(s).$$

2. **Text evidence.** This counts how many times a transformer-based skill NER model detects s in the free-text body of the resume (responsibilities, descriptions, etc.). We maintain a dictionary `text_skill_counts` and set

$$\text{text_evidence}(s) = \text{text_skill_counts}[s].$$

3. **Total evidence.** The total evidence for s is the sum of structured and text evidence:

$$\text{total_evidence}(s) =$$

$$\text{structured_evidence}(s) + \text{text_evidence}(s).$$

Here $N_{\text{structured}}(s)$ denotes the number of tokens in the structured resume fields (experience environments, project technologies, etc.) whose canonical form equals s .

This is converted to a base evidence score via a saturating function

$$\text{base_evidence_score}(s) = 1 - e^{-\alpha \cdot \text{total_evidence}(s)},$$

with $\alpha = \log 2$, so that one co-occurrence yields ≈ 0.5 , two yields ≈ 0.75 , etc.

We use an exponential saturating function for `base_evidence_score` so that it is 0 when there is no evidence, monotone increasing in `total_evidence(s)`, and exhibits diminishing returns for repeated mentions of the same skill. With $\alpha = \log 2$, a single co-occurrence already yields `base_evidence_score(s)  $\approx 0.5$` , two yield ≈ 0.75 , and further repetitions push the score closer to 1 without letting very long resumes dominate purely by repetition.

We also look at “**text-only**” skills: skills detected by the transformer that never appear in the declared skills section. They are treated symmetrically for evidence computation but are tagged as `text_only` origin.

8.2.3 Candidate-Level Legitimacy Score

Given all skill-level evidence, we define a candidate-level legitimacy score using simple, interpretable rules:

- **No skills anywhere:** if there is no declared skills section and the transformer finds nothing, set $L = 0.0$ and label the reason as `no_skills_anywhere`.

- **Declared but unsupported skills:** if there are declared skills but the total evidence for *all* of them is zero (never mentioned anywhere else), set $L = 0.4$ with reason `declared_but_no_evidence`. This penalizes pure “keyword stuffing”.
- **Only text-based skills:** if there is no declared skills section but the transformer finds skills in the body, set $L = 0.5$ with reason `text_skills_only`. This captures cases where skills are implied by experience but not listed.
- **Evidence-based case:** otherwise, for each skill s with some evidence we compute a final per-skill score

$$\text{final_skill_score}(s) =$$

$$0.5 + 0.5 \cdot \text{base_evidence_score}(s),$$

which lies in $(0.5, 1]$, and define L as the average of these final skill scores. Candidates who frequently mention their claimed skills in responsibilities and projects get scores closer to 1.0.

Alongside L we store metadata such as:

- total number of skills considered,
- whether the candidate had a skills section,
- whether the transformer found any text-based skills,
- total evidence counts,
- and a string reason explaining the dominant rule.

8.2.4 Storage in the Candidates Database

The final candidate-level outputs are materialized inside `candidates_db`:

- A table `candidate_skill_legitimacy` with columns:

```
(cand_id, legitimacy_score,
  num_skills_total,
  has_claimed_skills, has_text_skills,
  declared_total_evidence, reason).
```

This is keyed by `cand_id` and can be joined to any candidate query.

- A table `candidate_derived_skills(cand_id, skill_name, source)` which stores auto-extracted skills from projects, experience, and text-only transformer hits. These skills are not required to exist in `jobs_db.skills`; they serve as a richer signal for matching.

8.2.5 Integration into Federated Queries

At query time, the executor exposes two helper functions: one to fetch legitimacy rows and one to load derived skills for an arbitrary set of candidate IDs. These are integrated into several tasks inside `run_query.py` and the decomposition layer.

- **Skill-based candidate search.** In `filter_candidates_by_skills`, a candidate can match via explicit skills or via derived skills. For each match we attach:

- `legitimacy_score`,
- a bucket label
high/medium/low/very_low based
on thresholds (e.g., $L \geq 0.8$ is high),
- and the categorical reason.

The result list is then sorted primarily by number of matched skills and secondarily by L , so candidates whose skills are well supported in their resume body bubble to the top.

- **Candidate readiness.** In the federated `candidate_readiness` task, we compute overlap, coverage and Jaccard between the role skill set (from `jobs_db`) and each candidate’s skill set. Explicit skills drive the thresholds; derived skills are included as “bonus” evidence. Each candidate in the final list carries both the readiness metrics and the legitimacy fields, letting an analyst distinguish between, e.g., high-readiness but low-legitimacy profiles (possibly embellished) and slightly lower coverage but high-legitimacy profiles.
- **Multi-filter candidate queries.** In composite tasks that intersect multiple filters (skills, experience, location, projects), we merge candidate-level info from each subplan and then sort using a key that includes L after primary constraints. This means that among equally qualified candidates (same skills and experience filters), those with stronger evidence-backed skills are preferred.

Overall, this innovation upgrades our system from a purely keyword-based matcher to a *trust-aware* federated planner: results are still backed by SQL, but are now ordered and optionally gated by a principled, evidence-based measure of how believable each candidate's skills actually are.

9 Examples (with Snippets)

The Query Decomposer and Analyser can do queries like this — mixing SPJ, aggregates, and federated matches (only from our project's examples). For each, we show: the query, the task chosen by the parser, SQL decomposition, and the shape of the output our executor returns.

Example 1 — Federated readiness & hiring plan

Query. *“i am looking for people to build my web-site, for what job roles should i hire for and top 10 potential candidates”*

Task (parser). candidate_readiness (Aggregate, federated)

Decomposition. Two SQLs (jobs -> role skills, candidates -> skills & tenure)

```
-- SQL-1 (jobs_db): learn role skill set for
"web developer"
SELECT s.name, COUNT(*) AS freq
FROM jobs j
JOIN job_skills js ON j.job_id = js.job_id
JOIN skills s ON s.skill_id = js.skill_id
WHERE j.title_lc LIKE '%web developer%'
GROUP BY s.name
ORDER BY freq DESC
LIMIT 30;

-- SQL-2 (candidates_db): candidate skills +
tenure (robust date parsing)
WITH spans AS (
  SELECT e.cand_id AS cid,
    CASE
      WHEN e.start_date ~
        '^[0-9]{4}-[0-9]{2}-[0-9]{2}$' THEN
        e.start_date::date
      WHEN e.start_date ~
        '^[0-9]{4}-[0-9]{2}$' THEN
        to_date(e.start_date || '-01', 'YYYY-MM-DD')
      WHEN e.start_date ~ '^[0-9]{4}$'
      THEN to_date(e.start_date ||
        '-01-01', 'YYYY-MM-DD')
      ELSE NULL
    END AS sd,
    CASE
      WHEN NULLIF(e.end_date, '') IS NULL
      THEN NULL
      WHEN e.end_date ~
        '^[0-9]{4}-[0-9]{2}-[0-9]{2}$' THEN
        e.end_date::date
      WHEN e.end_date ~
        '^[0-9]{4}-[0-9]{2}$' THEN
        to_date(e.end_date || '-01', 'YYYY-MM-DD')
```

```
      WHEN e.end_date ~ '^[0-9]{4}$' THEN
        to_date(e.end_date || '-01-01', 'YYYY-MM-DD')
      ELSE NULL
    END AS ed
  FROM experience e
), spans_norm AS (
  SELECT cid, sd, COALESCE(ed, CURRENT_DATE) AS
    ed FROM spans
), agg AS (
  SELECT cid, SUM(GREATEST(0, (ed - sd))) /
    365.25 AS years
  FROM spans_norm WHERE sd IS NOT NULL GROUP BY
    cid
)
SELECT c.cand_id AS cid,
  COALESCE(MAX(a.years), 0.0) AS years,
  cs.skill_id, s.name,
  COALESCE(MAX(e.location_lc),
    MAX(c.location_lc)) AS loc,
  MAX(c.name) AS cname
FROM candidates c
LEFT JOIN agg a ON a.cid = c.cand_id
LEFT JOIN candidate_skills cs ON c.cand_id =
  cs.cand_id
LEFT JOIN skills s ON s.skill_id = cs.skill_id
LEFT JOIN experience e ON e.cand_id = c.cand_id
GROUP BY c.cand_id, cs.skill_id, s.name;
```

Structured result (shape; illustrative).

```
{
  "role_skills": ["html", "css", "javascript",
    "react", "git", "..."],
  "matching_candidates": [
    {
      "candidate_id": 123,
      "name": "Candidate 123",
      "location": "bengaluru, india",
      "overlap":
        ["javascript", "react", "css", "html"],
      "coverage": 0.35,
      "jaccard": 0.22,
      "skills":
        ["react", "css", "html", "git", "webpack",
          "node.js"]
    }
  ]
}
```

Summary (human-readable; formatter output).

```
Found 10 candidate(s) ready for web developer.
- Candidate 123 -- bengaluru, india:
  overlap=javascript, react, css, html
- Candidate 456 -- mumbai, india:
  overlap=javascript, css, html
Most frequent role skills: html, css,
  javascript, react, git, ...
```

Example 2 — Related skills (cluster-aware analytics)

Query. *“What skills are related to Kubernetes?”*

Task (parser). similar_skills_for_skill (Aggregate, jobs_only, cluster-aware)

Decomposition. Single SQL over jobs_db using skill_cluster_membership and

skill_clusters, plus a post-processing step that removes the base skill and returns the remaining skills in the same cluster ranked by job frequency.

```
WITH base_skill AS (
  SELECT skill_id, name_lc
  FROM skills
  WHERE name_lc LIKE '%kubernetes%'
  ORDER BY LENGTH(name_lc) ASC
  LIMIT 1
),
target_cluster AS (
  SELECT scm.skill_cluster_id
  FROM skill_cluster_membership scm
  JOIN base_skill b ON b.skill_id = scm.skill_id
)
SELECT
  sc.skill_cluster_id,
  sc.label,
  sc.size,
  s.skill_id,
  s.name_lc,
  COUNT(js.job_id) AS job_count
FROM target_cluster tc
JOIN skill_cluster_membership scm
  ON scm.skill_cluster_id = tc.skill_cluster_id
JOIN skills s
  ON s.skill_id = scm.skill_id
LEFT JOIN skill_clusters sc
  ON sc.skill_cluster_id = scm.skill_cluster_id
LEFT JOIN job_skills js
  ON js.skill_id = scm.skill_id
GROUP BY sc.skill_cluster_id, sc.label,
  sc.size, s.skill_id, s.name_lc
ORDER BY job_count DESC
LIMIT 20;
```

Structured result (shape; illustrative).

```
{
  "base_skill": "kubernetes",
  "cluster_id": 7,
  "cluster_label": "kubernetes docker helm",
  "cluster_size": 120,
  "related_skills": [
    {"skill_id": 101, "skill": "docker",
     "job_count": 51234},
    {"skill_id": 102, "skill": "helm",
     "job_count": 21345},
    {"skill_id": 103, "skill": "container
orchestration", "job_count": 18760},
    {"skill_id": 104, "skill": "microservices",
     "job_count": 15432},
    ...
  ]
}
```

Summary (human-readable; formatter out-put).

Skills related to Kubernetes (based on skill clusters and job co-occurrence):

- docker (appears in 51,234 jobs)
- helm (appears in 21,345 jobs)
- container orchestration (appears in 18,760 jobs)
- microservices (appears in 15,432 jobs)
- ...

Example 2 — Related skills

Query. “What skills are related to Kubernetes?”

Structured result (shape; illustrative).

```
{
  "base_skill": "Kubernetes",
  "cluster_id": 34,
  "cluster_label": "data",
  "cluster_size": 106513,
  "related_skills": [
    {"skill_id": 2333819, "skill":
"troubleshooting", "job_count": 41792},
    {"skill_id": 1893712, "skill": "python",
"job_count": 23687},
    {"skill_id": 2150717, "skill": "sql",
"job_count": 20788},
    {"skill_id": 1271341, "skill": "java",
"job_count": 13150},
    {"skill_id": 2060485, "skill": "security",
"job_count": 12338},
    {"skill_id": 309847, "skill": "aws",
"job_count": 11806},
    {"skill_id": 212994, "skill": "agile",
"job_count": 10802},
    {"skill_id": 583441, "skill": "computer
science", "job_count": 10292},
    {"skill_id": 410164, "skill": "c",
"job_count": 10023},
    {"skill_id": 2123239, "skill": "software
development", "job_count": 9888},
    {"skill_id": 2281050, "skill": "testing",
"job_count": 9102},
    {"skill_id": 1396141, "skill": "machine
learning", "job_count": 9076},
    {"skill_id": 1271789, "skill":
"javascript", "job_count": 8889},
    {"skill_id": 301724, "skill": "automation",
"job_count": 8586},
    {"skill_id": 691770, "skill": "data
analytics", "job_count": 8269}
  ]
}
```

Summary (human-readable; formatter out-put).

The result includes a base skill (“Kubernetes”), a cluster identifier, and a list of related skills with how many jobs mention each of them.

- Troubleshooting appears in over 41,000 jobs.
- Python (23,687 jobs), SQL (20,788 jobs), and Java (13,150 jobs) are also common.
- AWS (11,806 jobs), Security (12,338 jobs), Agile (10,802 jobs), and Software Development (9,888 jobs) are frequently associated.
- Machine Learning (9,076 jobs) and Data Analytics (8,269 jobs) also show a notable presence.

Example 3 — Jobs listing (SPJ)

Query. “Jobs listing for software engineer in London”

Task (parser). list_jobs_for_role

Decomposition.

```
SELECT j.job_id, j.title, j.company, j.location
FROM jobs j
WHERE j.title_lc LIKE '%software engineer%'
      AND (j.country_lc = 'united kingdom'
            OR j.location_lc ~*
            '(?i)(^[^a-z])london([^\a-z]|$)')
LIMIT 50;
```

Structured result (shape; illustrative).

```
[
  {"job_id": 9012, "title": "Software Engineer", "company": "Acme Ltd", "location": "London, UK"},
  {"job_id": 9013, "title": "Senior Software Engineer", "company": "Beta Inc", "location": "London, UK"}
]
```

Summary.

Top 10 jobs for software engineer in London (country-aware).

- Software Engineer -- Acme Ltd (London, UK)
- Senior Software Engineer -- Beta Inc (London, UK)

Example 4 — Eligible companies (explicit skills + location)

Query. “List companies I am eligible for as backend engineer in Europe skills: Java, Spring, Kubernetes.”

Task (parser).
eligible_companies_for_candidate (SPJ, jobs_only)

The regex block for eligible_companies_for_candidate picks up:

- role = “backend engineer” (from “as backend engineer”)
- location = “Europe” (from “in Europe”)
- skills_text = “Java, Spring, Kubernetes” (from “skills: ...”)

Decomposition. Single SQL over jobs_db, plus a deterministic ranking step:

```
-- SQL (jobs_db): fetch role-consistent postings
SELECT j.company, j.title, j.location, j.job_id
FROM jobs j
WHERE j.title_lc LIKE '%backend engineer%'
      AND (j.country_lc = 'europe'
            OR j.location_lc ~*
            '(?i)(^[^a-z])europe([^\a-z]|$)')
LIMIT 300;
```

In the executor, we tokenize the user-provided skills and compute a simple overlap score against

the textual job context (company, title, location) for ranking:

```
cskills = _split_skills_text("Java, Spring, Kubernetes")
base_set = cskills # here we do NOT fall back to role skills

scored = []
for company, title, location, job_id in rows:
    text = " ".join([str(company or ""), str(title or ""), str(location or "")]).lower()
    toks = set(_re.findall(r"[a-zA-Z0-9+#\.\.]+", text))
    hits = len(base_set & toks)
    scored.append({
        "company": company,
        "title": title,
        "location": location,
        "job_id": job_id,
        "overlap_hits": hits,
    })
scored.sort(key=lambda x: (x["overlap_hits"], (x["company"] or "")), reverse=True)
```

Structured result (shape; illustrative).

```
{
  "eligible_companies": [
    {
      "company": "Acme Cloud",
      "title": "Backend Engineer",
      "location": "Berlin, Germany",
      "job_id": 42137,
      "overlap_hits": 2
    },
    {
      "company": "Nordic Tech",
      "title": "Senior Backend Engineer",
      "location": "Stockholm, Sweden",
      "job_id": 42155,
      "overlap_hits": 1
    }
  ]
}
```

Summary (human-readable; formatter output).

Eligible backend engineer companies in Europe (ranked by skill-text overlap):

- Acme Cloud -- Backend Engineer (Berlin, Germany) [hits: 2]
- Nordic Tech -- Senior Backend Engineer (Stockholm, Sweden) [hits: 1]
- ...

Example 5 — Eligible companies (role + free-form skills)

Query. “What companies could I join as an Machine Learning engineer if I know Python, PyTorch, and Docker?”

Task (parser).

eligible_companies_for_candidate (SPJ, jobs_only)

We added a generic pattern for “what/which companies can/could I join as ...if I know ...”, which reuses the same task:

- role = "Machine Learning engineer"
- location = NULL (no explicit location)
- skills_text = "Python, PyTorch, and Docker"

Decomposition. The SQL template is the same as Example 4, now without a location filter:

```
-- SQL (jobs_db): ML engineer postings (global)
SELECT j.company, j.title, j.location, j.job_id
FROM jobs j
WHERE j.title_lc LIKE '%machine learning
engineer%'
LIMIT 300;
```

The executor reuses the same scoring function, now with the ML-skill set:

```
cskills = _split_skills_text("Python, PyTorch,
and Docker")
base_set = cskills # {"python", "pytorch",
"and docker"} after splitting

...
toks = set(_re.findall(r"[a-zA-Z0-9+#\.\.]+",
text)) # tokens from company/title/location
hits = len(base_set & toks)
...
```

Structured result (shape; illustrative).

```
{
  "eligible_companies": [
    {
      "company": "VisionAI Labs",
      "title": "Machine Learning Engineer",
      "location": "San Francisco, CA",
      "job_id": 9812,
      "overlap_hits": 1
    },
    {
      "company": "DeepCompute",
      "title": "ML Engineer",
      "location": "Remote",
      "job_id": 9834,
      "overlap_hits": 1
    }
  ]
}
```

Summary.

Companies you could join as a Machine Learning engineer (ranked by skill-text overlap):

- VisionAI Labs -- Machine Learning Engineer (San Francisco, CA)
- DeepCompute -- ML Engineer (Remote)
- ...